

INFO8006 Introduction to Artificial Intelligence

Exercise session 1: Solving problems by searching

RECAP

Search problem

A *search problem* is defined by

- the *states* $s \in \mathcal{S}$ and the *initial state* $s_1 \in \mathcal{S}$;
- the *actions* $\text{actions}(s) \subseteq \mathcal{A}$ available in state s ;
- the *transition model* $s' = \text{result}(s, a)$, the state reached by taking a in s ;
- the *goal states* $G \subseteq \mathcal{S}$, tested by the *goal test* $s \in G$;
- the *step costs* $c(s, a, s') > 0$ of taking a in s to reach s' .

Together, s_1 , actions and result define the *state space*, the set of states reachable from s_1 by any sequence of actions. A *solution* is an action sequence $a_{1:T}$ that leads from s_1 to a goal state, *i.e.* such that $s_{t+1} = \text{result}(s_t, a_t)$ for $t = 1, \dots, T$ and $s_{T+1} \in G$. Its *path cost* is

$$\sum_{t=1}^T c(s_t, a_t, s_{t+1}),$$

and an *optimal solution* is a solution of minimal path cost.

Search algorithms

TREE-SEARCH expands the nodes n of the *fringe*, each a path from s_1 to a state $s(n)$, until a goal state leaves it. GRAPH-SEARCH also keeps the set of expanded states, the *closed* set, which avoids repeated states and cycles.

```
function TREE-SEARCH(problem, fringe) returns a solution, or failure
  fringe  $\leftarrow$  INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node  $\leftarrow$  REMOVE-FRONT(fringe)
    if GOAL-TEST(problem, STATE[node]) then return node
    fringe  $\leftarrow$  INSERTALL(EXPAND(node, problem), fringe)
```

```
function GRAPH-SEARCH(problem, fringe) returns a solution, or failure
  closed  $\leftarrow$  an empty set
  fringe  $\leftarrow$  INSERT(MAKE-NODE(INITIAL-STATE[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node  $\leftarrow$  REMOVE-FRONT(fringe)
    if GOAL-TEST(problem, STATE[node]) then return node
    if STATE[node] is not in closed then
      add STATE[node] to closed
      fringe  $\leftarrow$  INSERTALL(EXPAND(node, problem), fringe)
  end
```

Search strategies

A *strategy* is the order in which nodes leave the fringe, based on the path cost $g(n)$ from s_1 to n , a heuristic $h(n)$, or $f(n) = g(n) + h(n)$, the estimated cost of the cheapest solution through n . In the table below, b is the maximum branching factor, d the depth of the least-cost solution, m the maximum path length (possibly infinite), C^* the optimal path cost and $\epsilon > 0$ a lower bound on the step costs.

Strategy	Fringe	Complete	Optimal	Time	Space
Depth-first (DFS)	LIFO stack	if no cycles	no	$O(b^m)$	$O(bm)$
Breadth-first (BFS)	FIFO queue	yes	if c uniform	$O(b^d)$	$O(b^d)$
Iterative deepening	DFS, depth $1, 2, \dots$	yes	if c uniform	$O(b^d)$	$O(bd)$
Uniform-cost (UCS)	priority $g(n)$	if $c \geq \epsilon$	yes	$O(b^{C^*/\epsilon})$	$O(b^{C^*/\epsilon})$
Greedy	priority $h(n)$	if no cycles	no	$O(b^m)$	$O(b^m)$
A*	priority $f(n)$	yes	h admissible (tree)	$O(b^m)$	$O(b^m)$

Heuristics

A *heuristic* $h(n) \geq 0$ estimates the true cost $h^*(n)$ of the cheapest path from n to a goal state. It is zero at goal states, and is often the exact cost of a *relaxed problem*, obtained by removing constraints from the original one.

- *Admissible*: $h(n) \leq h^*(n)$ for all n , i.e. h never overestimates. A* with TREE-SEARCH is then optimal.
- *Consistent*: $h(n) \leq c(n, a, n') + h(n')$ for every node n and every successor n' generated by an action a . A consistent heuristic is admissible, f never decreases along a path, and A* with GRAPH-SEARCH is then optimal.
- *Dominance*: if h_1 and h_2 are admissible and $h_2(n) \geq h_1(n)$ for all n , then h_2 dominates h_1 : it is closer to h^* , and A* expands no more nodes with it. The maximum $\max(h_1, h_2)$ of two admissible heuristics is admissible and dominates both.

In session: exercises 1, 3 (a)–(c), 5.

Exercise 1 Path finding

Tram'Ardent maintains the green areas along the Liège tram line with a new robot. This robot's goal is to convey the grass cuttings left by the gardeners to a container as fast as possible. Assuming that

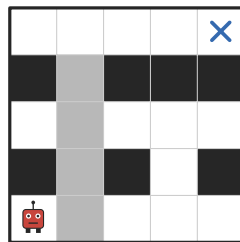
- the fuel cost used by the robot is negligible
- the robot can only navigate on guides that are placed in a rectangular grid configuration and can locate itself on the grid
- there is only one pile and one container by zone (*i.e.* by grid) for which positions are known by the robot
- the speed of the robot is constant and the grass loading/unloading is instantaneous
- the robot can only carry a limited weight of grass at once
- the robot can know the initial weight of grass on the pile.

For a given grid

- (a) Formulate a corresponding search problem.
- (b) The terrain is rugged at some places, where the robot slows down: a step into a rugged cell costs more than 1, the cost of a step on flat terrain. Propose an admissible heuristic, which estimates the remaining cost at any node.
- (c) The robot can now also move diagonally, a diagonal step costing $\sqrt{2}$. Is your heuristic still admissible? If not, what should you change? What if the robot is twice as fast diagonally, so that a diagonal step costs $\sqrt{2}/2$?

Exercise 2 Sewer inspection (CS188, Spring 2019)

You control an inspection robot in a rectangular maze of sewers of size $M \times N$, as shown in the figure below. At each time-step, the robot can move into a free adjacent cell or stay in its current location. All actions have a unit cost.

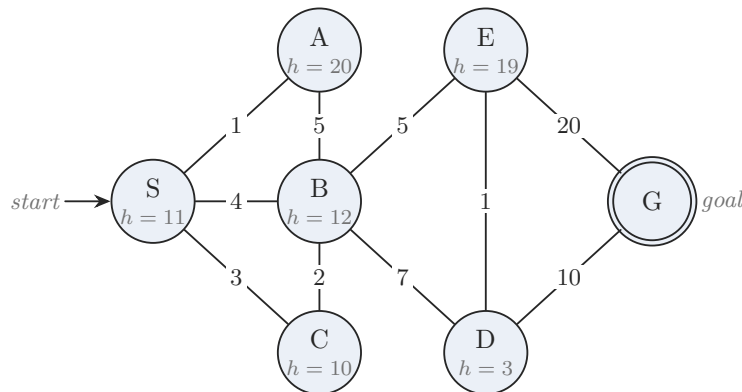


In this particular case, the robot must pass through a series of partially flooded tunnels, as illustrated by the gray cells on the map. In a flooded cell, the robot runs on a sealed backup battery, which lasts A time-steps in a row. Moving into a flooded cell uses 1 unit of that battery, while moving into a free cell recharges it completely.

- (a) Give a minimal state space for this problem (do not include extra information). You should answer for a general instance of the problem, not the specific map above.
- (b) Give the size of your state space.

Exercise 3 Route planning

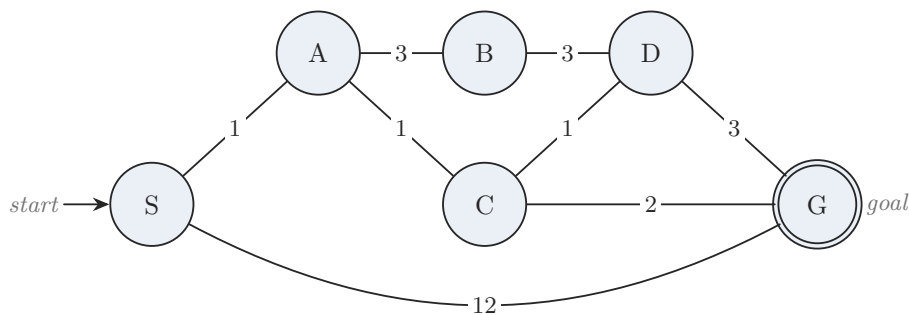
A navigation app plans a route from junction S to junction G on the road network drawn below, whose edges are labelled with travel times in minutes. The heuristic value h of each junction, on its vertex, is the travel time to G predicted by a learned model. For each of the following search algorithms, apply the GRAPH-SEARCH algorithm of the recap to this undirected graph, and give the order in which the states are expanded, as well as the final path returned by the algorithm. Ties are broken by alphabetical order on the states and, between two nodes of the same state, in favour of the one inserted first in the fringe.



- Depth-first search (DFS)
- Breadth-first search (BFS)
- Uniform-cost search (UCS)
- Greedy search
- A*
 - Is the heuristic admissible? If not, make it admissible.
 - Is the heuristic consistent? If not, can we apply GRAPH-SEARCH?
 - Apply A* with the admissible heuristic, using the variant of GRAPH-SEARCH that re-expands a closed state reached with a lower priority.

Exercise 4 Pathfinding in a video game

A character of a video game, controlled by the game, moves from S to G between the waypoints of a level, drawn below with the lengths of the passages between them; the direct passage from S to G is a long one. Apply the GRAPH-SEARCH algorithm of the recap to this graph, with the tie-breaking of exercise 3.



- Apply iterative deepening, with a depth limit growing from 1. In which order are the states expanded, which path is returned, and is it optimal?

- (b) The designers of the level propose two estimates of the distance to G:

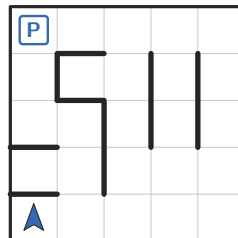
State	h_1	h_2
S	5	4
A	3	2
B	6	6
C	2	1
D	3	3
G	0	0

which one is not admissible? Why?

- (c) Is the admissible heuristic consistent? Does it matter here?
- (d) Apply A* with the admissible heuristic.
- (e) Which of these strategies are guaranteed to return the optimal path on this graph?

Exercise 5 Self-parking car (CS188, Spring 2014)

A self-parking car must reach a free parking spot in a car park, seen from above as a grid with walls. At all time-steps, the car points at direction $d \in \{N, S, E, W\}$. With a single action, the car can either move forward at an adjustable velocity $v \in [0, V]$ or turn. The turning actions are **left** and **right**, which change the car's direction by 90 degrees. Turning is only permitted when the velocity is zero (and leaves it at zero). The moving actions are **faster** and **slower**. The action **faster** increments the velocity by 1 and **slower** decrements the velocity by 1; in both cases the car then moves a number of squares equal to its *new* velocity. There is no action that keeps the velocity: the car cannot move at the same nonzero velocity for two consecutive time steps. Any action whose move would cross a wall is illegal, whichever square the car stops on. Any action that would take v below 0 or above a maximum speed V is also illegal. A car moving too fast towards a wall may therefore be left with no legal action. The goal is a plan which parks the car (stationary) on the free spot using as few actions (time steps) as possible.



As an example, in the car park above, if the car is initially stationary, it might first turn to the east using **right**, then move one square east using **faster**, then two more squares east using **faster** again. However, the car will have to slow to take the turn.

- (a) For a grid of size $M \times N$, what is the size of the state space? Assume that all configurations are reachable from the starting state.
- (b) What is the maximum branching factor of this problem? Assume that illegal actions are simply not in $\text{actions}(s)$.
- (c) Is the Manhattan distance from the car's location to the free spot admissible?
- (d) If we used an inadmissible heuristic in A* tree search, could it change the completeness of the search? And the optimality?
- (e) State and motivate a non-trivial admissible heuristic for this problem.
- (f) Give a general advantage that an inadmissible heuristic might have over an admissible one.

Exercise 6 Learned heuristic (CS188, Spring 2019)

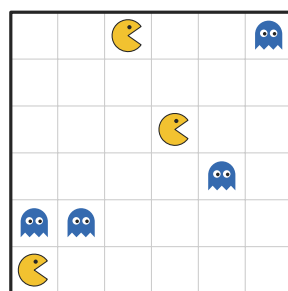
Heuristics are increasingly learned: a neural network, trained on solved instances of a problem, predicts the remaining cost of a node. Consider a graph search problem where all edges have a unit cost and the optimal solution has cost C^* , and a network calibrated to never overestimate, which predicts $h(n) = \max\{h^*(n) - k, 0\}$, where $h^*(n)$ is the true forward cost of n and $k \leq C^*$ is a non-negative constant.

- (a) Is h admissible?
- (b) Which of the following is the most reasonable description of how much more work will be done, *i.e.* how many more nodes will be expanded, with heuristic h compared to h^* , as a function of k ?
- ☐ Constant in k
 - ☐ Linear in k
 - ☐ Exponential in k
 - ☐ Unbounded

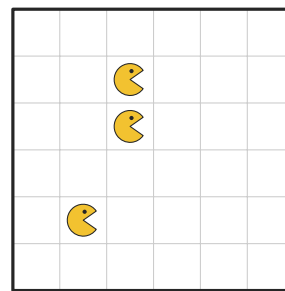
Exercise 7 Pacmen (Exam, January 2023)

Pacman and his friends have decided to combine forces and go on the offensive, and are now chasing ghosts! In a grid of size $M \times N$, Pacman and $P - 1$ of his friends are moving around to collectively eliminate all of the ghosts in the grid by stepping on the same square as each of them. Moving into the same square as a ghost will eliminate it from the grid. At every turn, Pacman and his friends may choose one of the four (**north**, **south**, **east**, **west**) actions but may not collide with each other. In other words, any action that would result in two or more Pacmen occupying the same square will result in no movement for either of the Pacmen. Additionally, Pacman and his friends are *indistinguishable* from each other. There are a total of G ghosts, which are also indistinguishable from each other and cannot move.

Treating this as a search problem, we consider each configuration of the grid to be a state, and the goal state to be the configuration where all ghosts have been eliminated from the board. The cost of each turn (all Pacmen move once) is 1. Below is an example starting state, as well as an example of goal state.



Initial state example



Final state example

Suppose that Pacman has no friends ($P = 1$).

- (a) What is the size of the minimal state space representation? Explain your answer.
- (b) Explain whether each of the following heuristics is admissible, consistent, neither, or both.
- $h_1(n)$ = the sum of the Manhattan distances from Pacman to every remaining ghost.
 - $h_2(n)$ = the number of ghosts times the maximum Manhattan distance between Pacman and any of the ghosts.
 - $h_3(n)$ = the number of remaining ghosts.

Suppose that Pacman has exactly one less friend than the number of ghosts ($P = G$).

- (c) What is the size of the minimal state space representation? Explain your answer.
- (d) Explain whether each of the following heuristics is admissible, consistent, neither, or both.
- $h_4(n)$ = the largest of the Manhattan distances between each Pacman and its closest ghost.
 - $h_5(n)$ = the smallest of the Manhattan distances between each Pacman and its closest ghost.
 - $h_6(n)$ = the number of remaining ghosts.
 - $h_7(n)$ = the number of remaining ghosts divided by P .

QUIZ

1. An agent is rational if ...
 - ☐ its actions always lead to the best possible outcome.
 - ☐ it knows the state s_t of the environment at all times.
 - ☐ its policy maximizes $\mathbb{E}[V(s_{1:T}) \mid \pi]$, given its percepts and prior knowledge.
 - ☐ it maximizes the performance measure that its designer had in mind.
2. As a task environment, poker is ...
 - ☐ discrete, multi-agent and deterministic.
 - ☐ partially observable, stochastic and multi-agent.
 - ☐ partially observable, episodic and single-agent.
 - ☐ None of the above.
3. In a stochastic environment, a simple reflex agent ($a_t = \pi(e_t)$) given the same percept twice will ...
 - ☐ take the same action every time.
 - ☐ take a different action every time.
 - ☐ take a different action most of the time.
 - ☐ take the same action most of the time.
4. A goal-based agent ...
 - ☐ selects its action from the current percept e_t only.
 - ☐ searches its transition model for an action sequence that reaches a state of G .
 - ☐ prefers a fast and safe route over a slow and risky one.
 - ☐ maximizes the expected performance $\mathbb{E}[V(s_{1:T}) \mid e_{1:t}, a_t]$ of its actions.
5. The A^* algorithm, in its graph version (GRAPH-SEARCH, without re-expansion) is ...
 - ☐ always optimal.
 - ☐ always optimal if the heuristic is admissible.
 - ☐ always optimal if the heuristic is consistent.
 - ☐ never optimal when there are cycles.
6. Which of the following statements about heuristics is true?
 - ☐ $h(n) = 0 \ \forall n$ can speed up A^* compared to UCS.
 - ☐ If a heuristic is admissible, it is guaranteed to be consistent.
 - ☐ The average step cost $h(n) = \frac{1}{|\text{actions}(s)|} \sum_a c(s, a, \text{result}(s, a))$, with $s = s(n)$, is admissible.
 - ☐ The true remaining cost dominates all admissible heuristics.

7. Iterative deepening is often preferred to breadth-first search because it ...
- ☐ expands strictly fewer nodes.
 - ☐ finds a solution at the same depth d with $O(bd)$ memory, instead of $O(b^d)$.
 - ☐ is optimal even when the step costs differ.
 - ☐ never expands the same state twice.
8. In GRAPH-SEARCH, keeping a closed set ...
- ☐ makes depth-first search complete on a finite state space.
 - ☐ makes any admissible heuristic consistent.
 - ☐ guarantees that A^* is optimal whenever h is admissible.
 - ☐ is only useful when the state space is infinite.